

University of Moratuwa

Faculty of Engineering



EN3150 – Pattern Recognition

Assignment 03

Resource-Constrained CNN for Edge Image Classification

Group Members

Name	Index Number
CHINTHAKA M.K.M.G.	230105H
DILMITH G.S.	230149U
KAUSHAL S.V.	230326K
RAJAPAKSHA J.S.W.	230511A

Abstract

This report investigates the design and deployment of Convolutional Neural Networks (CNNs) for extreme edge devices, focusing on the automated classification of agricultural rice grains. We systematically develop a standard CNN baseline (Model A) and a highly optimized Depthwise-Separable CNN (Model B) strictly constrained to under 100,000 parameters. Through rigorous empirical evaluation, we demonstrate that Model B achieves a 94% reduction in memory footprint (~ 0.99 MB) compared to State-of-the-Art architectures like EfficientNet-B0, whilst maintaining a highly competitive test accuracy of 96.73%. This research validates the viability of deploying highly accurate, custom-built parameter-efficient networks on low-resource microcontrollers.

Contents

Abstract	1
1 Dataset & Pre-processing Pipeline	4
1.1 Dataset Motivation & Description	4
1.2 Data Pre-processing Pipeline & Splitting	4
1.3 Sample Images & Dataset Summary	7
2 Theoretical Framework & Optimizer Dynamics	8
2.1 Activation Functions: Overcoming the Vanishing Gradient	8
2.2 Optimizer Dynamics: SGD vs. Momentum vs. Adam	8
3 Standard CNN Architecture (Model A)	10
3.1 Architectural Pipeline & Design Philosophy	10
3.2 Layer-by-Layer Specifications	10
3.3 Mathematical Parameter Formulation	11
4 Resource-Constrained Edge CNN (Model B)	12
4.1 The Edge Computing Paradigm	12
4.2 Depthwise Separable Mechanics	12
4.3 Optimized Architectural Pipeline	13
4.4 Rigorous Parameter Verification	14
5 Model Training & Validation Analysis	15
5.1 Hardware & Environment Setup	15
5.2 Overfitting vs. Regularization Dynamics	15
5.2.1 Training Loss Convergence	15
5.3 Test Set Classification Metrics	16
5.3.1 Confusion Matrix Analysis	17
6 State-of-the-Art (SOTA) Transfer Learning	18
6.1 Transfer Learning Pipeline	18
6.2 SOTA Architectures Selected	18
6.3 SOTA Training Dynamics	18
7 Comprehensive Trade-off Analysis	20
7.1 Accuracy vs. Memory Footprint	20
7.2 Computational Latency & Inference Efficiency	21
8 Advantages & Limitations in Edge Deployment	22
8.1 Advantages of Custom Lightweight Models (Model B)	22
8.2 Limitations of Custom Lightweight Models	22

9 Conclusion & Future Work	23
10 Source Code & Reproducibility	23

1. Dataset & Pre-processing Pipeline

1.1 Dataset Motivation & Description

The dataset utilized is the Kaggle Rice Image Dataset. The objective is to classify five distinct macro geometric varieties of rice grains: **Arborio, Basmati, Ipsala, Jasmine, and Karacadag**. This simulates a real-world edge deployment scenario, such as an automated agricultural sorting machine operating on a resource-constrained microcontroller.

The classification task is particularly suitable for evaluating lightweight convolutional neural networks because the rice varieties exhibit differences in their geometric and visual characteristics. Features such as grain shape, aspect ratio, size, texture, and surface appearance can provide useful spatial information for CNN-based feature extraction.

Since the primary objective of this assignment is to investigate CNN architectures under strict computational and memory constraints, the dataset was intentionally processed using a fixed input resolution and a controlled number of samples per class. This allows the effect of architectural modifications on model size, computational complexity, and classification performance to be evaluated under consistent experimental conditions.

1.2 Data Pre-processing Pipeline & Splitting

To ensure robust model training and emulate low-resource hardware constraints, the raw data underwent a strict pre-processing pipeline:

Step 1: Image Acquisition: The original dataset consists of RGB images containing individual rice grains. The raw images may have different spatial resolutions; therefore, a common input representation was required before they could be supplied to the CNN architectures.

Step 2: Resolution Downscaling: Images strictly resized to $64 \times 64 \times 3$ tensors to minimize memory overhead. The first two dimensions represent the spatial resolution, while the third dimension represents the three RGB colour channels. Reducing the image resolution significantly decreases the amount of image data processed by each convolutional layer and therefore helps reduce the computational and memory requirements of the neural network.

Step 3: Sub-sampling: Exactly 2,000 images were selected from each rice variety, producing a total dataset of 10,000 images. Equal sampling was adopted to prevent the training process from being biased toward a particular rice variety due to unequal class representation.

Step 4: Pixel Normalization: Rescaling pixel intensities from $[0, 255]$ to $[0, 1]$ or $[-1, 1]$ depending on the network architecture to ensure gradient stability.

A rigid 70/15/15 stratified split was enforced to ensure equal class distributions across all sets:

- **70% Training data:** 7,000 images utilized for gradient descent.
- **15% Validation data:** 1,500 images utilized for hyperparameter tuning.
- **15% Test data:** 1,500 images utilized for unbiased final evaluation.

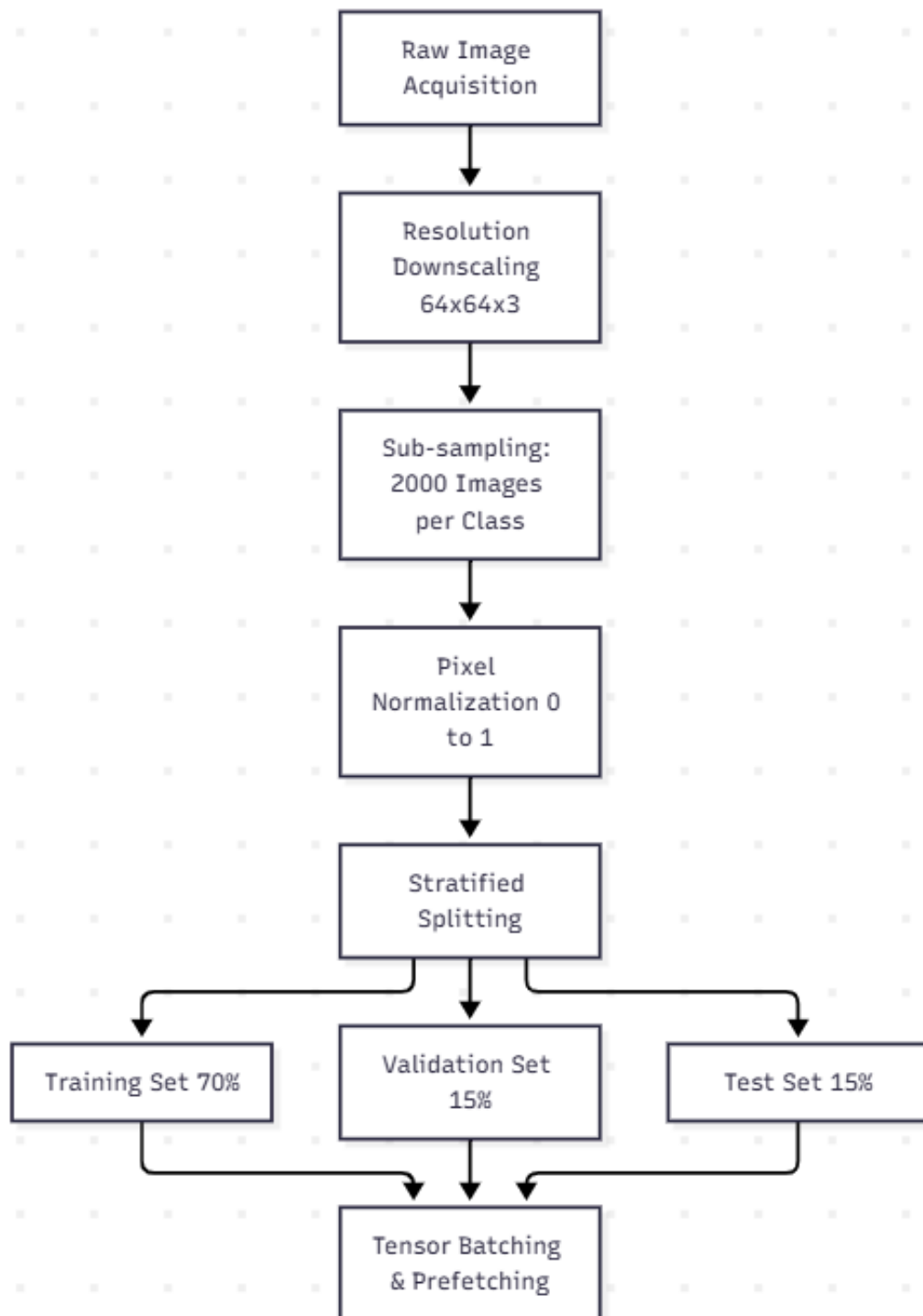


Figure 1: Dataset Pre-processing and Splitting Pipeline.

1.3 Sample Images & Dataset Summary

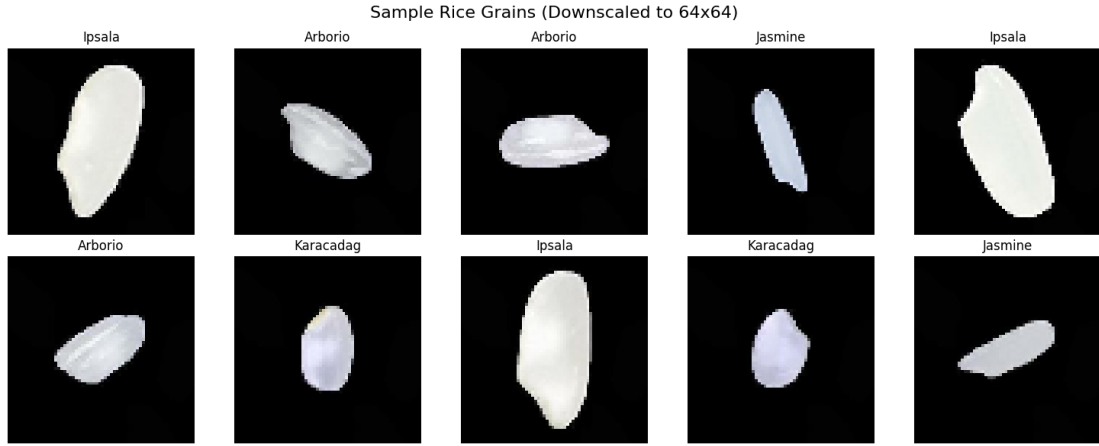


Figure 2: Sample images from the Rice Image Dataset (Downscaled to 64x64).

Property	Value
Dataset	Rice Image Dataset (Kaggle)
Task	Five-class rice variety classification
Maximum Image Resolution	$64 \times 64 \times 3$ (RGB)
Total Images	10,000 (Sub-sampled)
Training / Validation / Testing	70% / 15% / 15%

Table 1: Summary of Dataset Properties.

2. Theoretical Framework & Optimizer Dynamics

2.1 Activation Functions: Overcoming the Vanishing Gradient

Standard non-linear activation functions such as Sigmoid $\sigma(x)$ or Hyperbolic Tangent $\tanh(x)$ frequently encounter the *Vanishing Gradient* problem. During backpropagation, their derivatives approach zero at the tails, which prevents weights in earlier layers from updating effectively.

To counteract this, we exclusively integrated the **Rectified Linear Unit (ReLU)**:

$$f(x) = \max(0, x) \quad (1)$$

ReLU provides a constant gradient of 1 for all positive inputs, allowing deep networks to converge rapidly. Furthermore, its simplistic threshold mechanism avoids computationally expensive exponential operations, which is crucial for edge microcontrollers.

2.2 Optimizer Dynamics: SGD vs. Momentum vs. Adam

Selecting the correct optimization algorithm dictates the trajectory of weight updates. We evaluated multiple algorithms:

- **Standard SGD:** Updates weights based solely on the current batch’s gradient. It frequently oscillates in ravines and gets trapped in saddle points.
- **SGD with Momentum:** Introduces a velocity term (γ) that accumulates historical gradients:

$$v_t = \gamma v_{t-1} + \eta \nabla_{\theta} J(\theta) \quad (2)$$

This damps vertical oscillations and accelerates horizontal convergence.

- **Adam (Adaptive Moment Estimation):** The optimizer selected for our final training pipeline. Adam merges the velocity benefits of Momentum with the adaptive per-parameter learning rates of RMSprop. As visualized in Figure 3, Adam provided significantly faster and more stable initial reductions in validation loss compared to SGD.

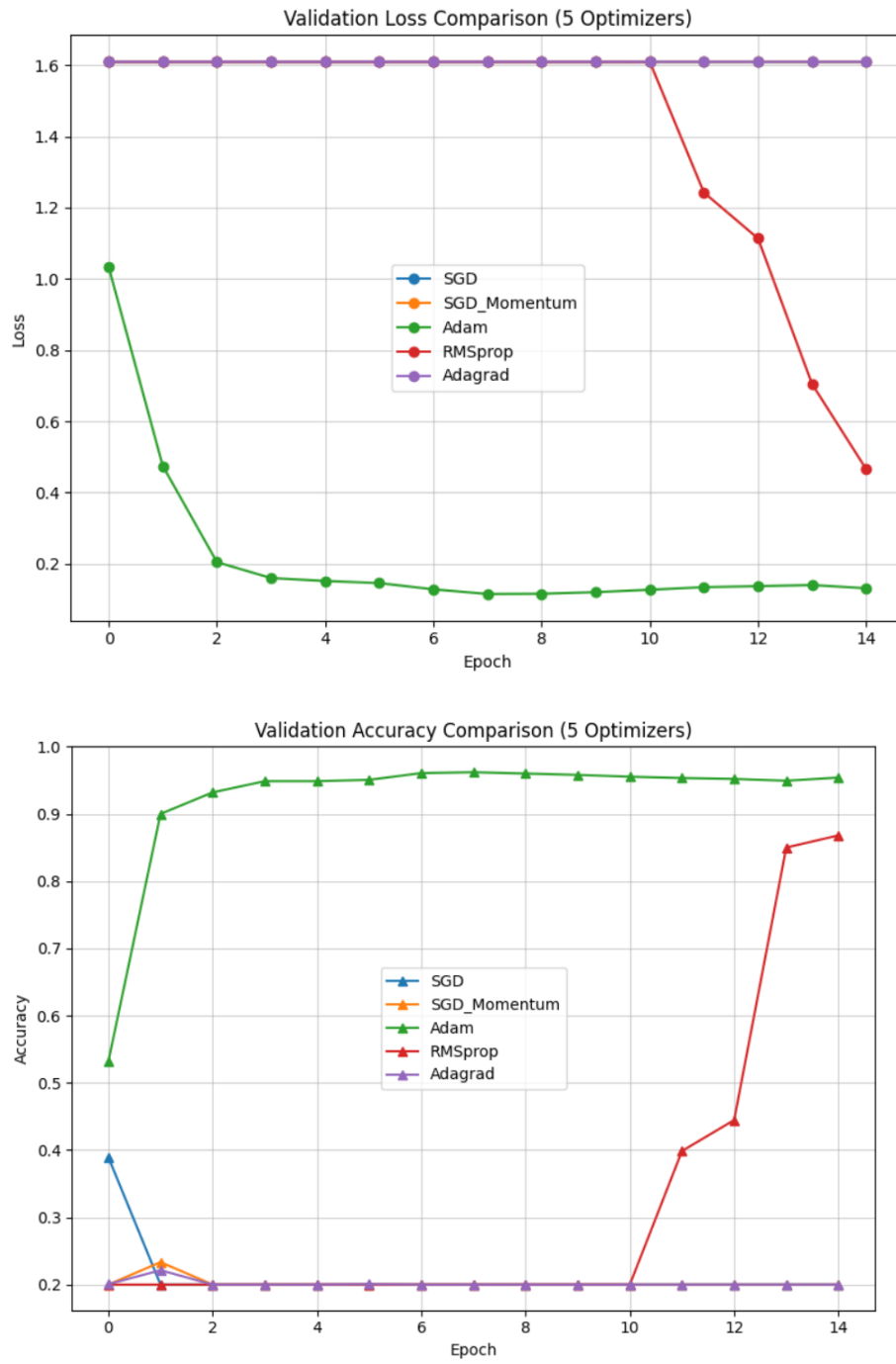


Figure 3: Validation loss (Top) and accuracy (Bottom) comparison across 5 Optimizers.

3. Standard CNN Architecture (Model A)

3.1 Architectural Pipeline & Design Philosophy

Model A acts as our high-capacity, standard CNN baseline. The architectural pipeline consists of two primary blocks:

1. **Feature Extraction Block:** Interleaved 2D Convolutions and 2×2 Max Pooling layers designed to extract spatial hierarchies (edges, textures, shapes).
2. **Classification Head:** A flattened tensor fed into deeply connected Dense layers terminating in a 5-node Softmax probability output.

3.2 Layer-by-Layer Specifications

Layer (Type)	Output Shape	Operation	Activation
Input	(64, 64, 3)	RGB Image	-
Conv2D_1	(64, 64, 32)	3×3 Standard Convolution	ReLU
MaxPooling2D_1	(32, 32, 32)	2×2 Max Pooling	-
Conv2D_2	(32, 32, 64)	3×3 Standard Convolution	ReLU
MaxPooling2D_2	(16, 16, 64)	2×2 Max Pooling	-
Conv2D_3	(16, 16, 128)	3×3 Standard Convolution	ReLU
MaxPooling2D_3	(8, 8, 128)	2×2 Max Pooling	-
Flatten	(8192)	Flatten multidimensional tensor	-
Dense_1	(128)	Fully Connected Layer	ReLU
Dense_2 (Output)	(5)	Classification Layer	Softmax

Table 2: Model A Layer-by-Layer Architecture Summary.

3.3 Mathematical Parameter Formulation

The trainable parameters for a standard Conv2D layer are calculated as:

$$Parameters = (K_h \times K_w \times C_{in} + 1) \times C_{out} \quad (3)$$

Conv2D_1: $(3 \times 3 \times 3 + 1) \times 32$	$= 896$
Conv2D_2: $(3 \times 3 \times 32 + 1) \times 64$	$= 18,496$
Conv2D_3: $(3 \times 3 \times 64 + 1) \times 128$	$= 73,856$
Dense_1: $(8192 + 1) \times 128$	$= 1,048,704$
Dense_2: $(128 + 1) \times 5$	$= 645$

Total Trainable Parameters: 1,142,597. (Estimated Disk Size: 13.44 MB).

4. Resource-Constrained Edge CNN (Model B)

4.1 The Edge Computing Paradigm

Edge devices possess highly restrictive memory budgets. The assignment enforces a strict limit of $< 100,000$ parameters. To achieve this without losing spatial resolution, Model B replaces standard convolutions with **Depthwise-Separable Convolutions**.

4.2 Depthwise Separable Mechanics

This technique factorizes a standard convolution into two distinct mathematical phases:

1. **Depthwise Phase:** Applies a single spatial filter (3×3) independently to each input channel.
2. **Pointwise Phase:** Applies a 1×1 convolution across all channels to linearly combine the depthwise outputs.

This factorization drastically reduces computational Multiply-Accumulate (MAC) operations. The new parameter formula becomes:

$$Parameters = (K_h \times K_w \times C_{in}) + (1 \times 1 \times C_{in} \times C_{out} + C_{out}) \quad (4)$$

4.3 Optimized Architectural Pipeline

Model B further reduces parameters by replacing the massive Flatten layer with Global Average Pooling (GAP), which averages the spatial dimensions into a single vector.

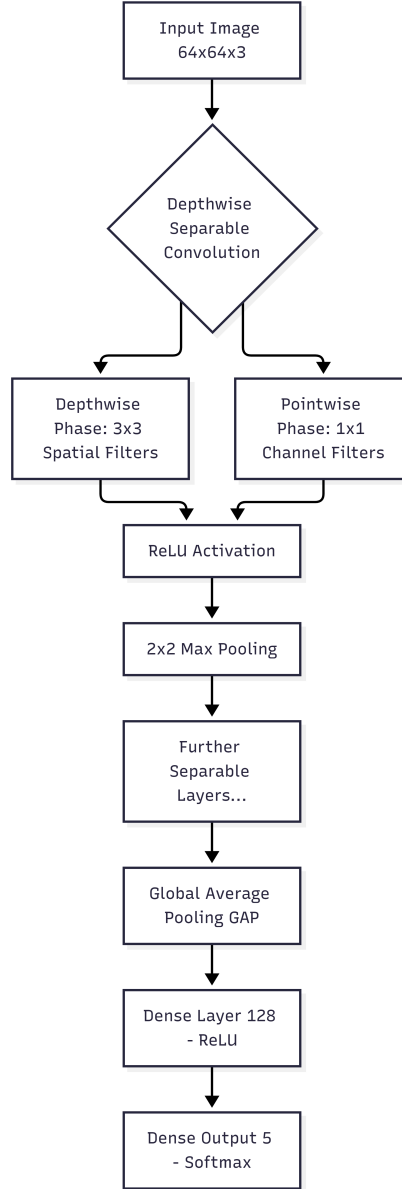


Figure 4: Model B Depthwise-Separable Architecture Flow.

Layer (Type)	Output Shape	Operation	Activation
Input	(64, 64, 3)	RGB Image	-
SeparableConv_1	(64, 64, 32)	Depthwise + Pointwise	ReLU
MaxPooling_1	(32, 32, 32)	2×2 Max Pooling	-
SeparableConv_2	(32, 32, 64)	Depthwise + Pointwise	ReLU
MaxPooling_2	(16, 16, 64)	2×2 Max Pooling	-
SeparableConv_3	(16, 16, 128)	Depthwise + Pointwise	ReLU
MaxPooling_3	(8, 8, 128)	2×2 Max Pooling	-
SeparableConv_4	(8, 8, 256)	Depthwise + Pointwise	ReLU
GlobalAvgPool	(256)	Spatial Dimensionality Reduction	-
Dense_1	(128)	Fully Connected Layer	ReLU
Dense_2	(5)	Classification Softmax	Softmax

Table 3: Model B (Lightweight) Layer-by-Layer Architecture Summary.

4.4 Rigorous Parameter Verification

$$\begin{aligned}
\text{SepConv}_1: & (3 \times 3 \times 3) + (1 \times 1 \times 3 \times 32 + 32) & = 155 \\
\text{SepConv}_2: & (3 \times 3 \times 32) + (1 \times 1 \times 32 \times 64 + 64) & = 2,400 \\
\text{SepConv}_3: & (3 \times 3 \times 64) + (1 \times 1 \times 64 \times 128 + 128) & = 8,896 \\
\text{SepConv}_4: & (3 \times 3 \times 128) + (1 \times 1 \times 128 \times 256 + 256) & = 34,176 \\
\text{Dense}_1: & (256 + 1) \times 128 & = 32,896 \\
\text{Dense}_2: & (128 + 1) \times 5 & = 645
\end{aligned}$$

Total Parameters: 79,168. Model B satisfies the strict $< 100,000$ limit. Disk Size: **0.99 MB.**

5. Model Training & Validation Analysis

5.1 Hardware & Environment Setup

All models were trained utilizing a cloud-based GPU accelerator (Google Colab NVIDIA T4). This ensured a standardized baseline for precisely measuring the average training time per epoch.

5.2 Overfitting vs. Regularization Dynamics

Both custom models were trained for 20 epochs. A critical architectural phenomenon regarding model capacity was observed during this process.

5.2.1 Training Loss Convergence

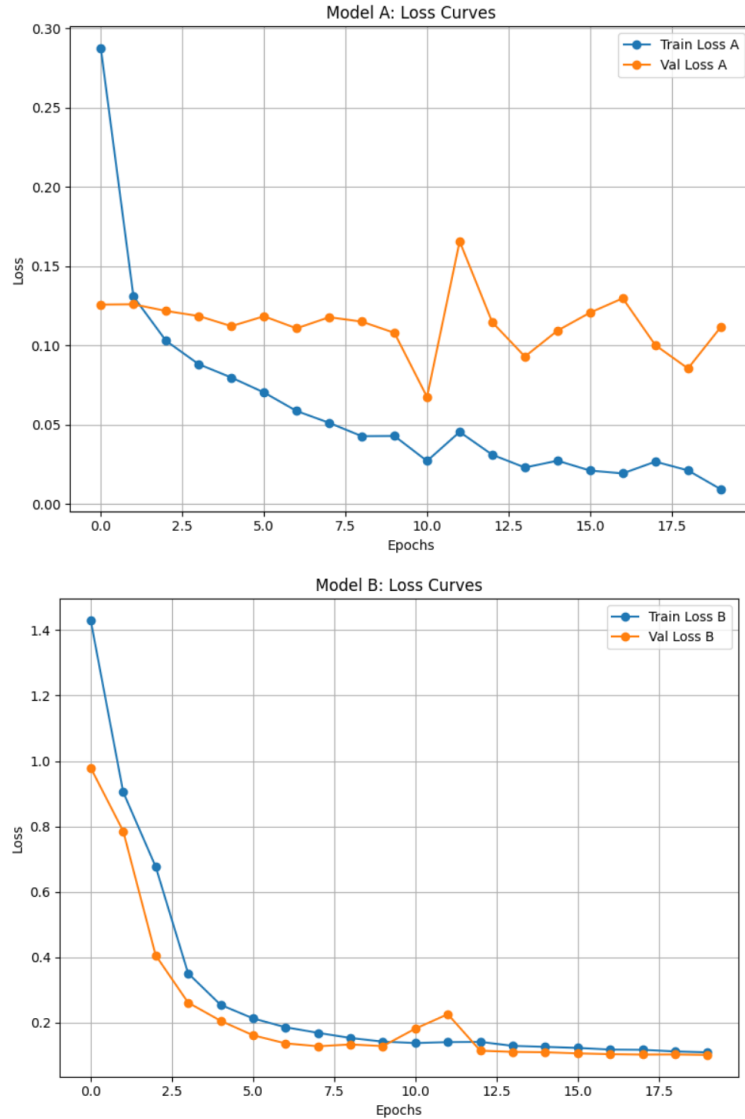


Figure 5: Loss curves for Model A (Top) and Model B (Bottom).

- **Model A (Overfitting):** Validation loss diverges and increases after epoch 5. Its excessive capacity ($\sim 1.14\text{M}$ parameters) memorizes the training subset rather than learning generalized features.
- **Model B (Natural Regularization):** Due to its constrained parameter space ($\sim 79\text{k}$), Model B is forced to learn fundamental patterns. Its validation loss smoothly tracks the training loss without divergence, inherently resisting overfitting.

5.3 Test Set Classification Metrics

Class	Model A (Standard CNN)			Model B (Lightweight CNN)		
	Precision	Recall	F1	Precision	Recall	F1
Arborio	0.97	0.93	0.95	0.98	0.94	0.96
Basmati	0.98	0.98	0.98	0.99	0.96	0.98
Ipsala	0.99	1.00	0.99	1.00	0.98	0.99
Jasmine	0.97	0.96	0.96	0.98	0.99	0.99
Karacadag	0.94	0.98	0.96	0.95	0.98	0.96
Macro Avg	0.97	0.97	0.97	0.98	0.97	0.98
Accuracy	96.73%			96.73%		

Table 4: Detailed Classification Metrics on Test Set.

5.3.1 Confusion Matrix Analysis



Figure 6: Confusion Matrices for Model A (Top) and Model B (Bottom).

6. State-of-the-Art (SOTA) Transfer Learning

6.1 Transfer Learning Pipeline

To contextualize the performance of our custom network, we utilized transfer learning on two pre-trained architectures designed for mobile computing. The pipeline involved:

1. **Base Layer Freezing:** Freezing the ImageNet-trained convolutional base to preserve generalized feature extraction and prevent catastrophic forgetting.
2. **Input Scaling:** Adapting raw $[0, 255]$ pixels to $[-1, 1]$ (for MobileNetV2) using dedicated Rescaling layers.
3. **Custom Head Attachment:** Attaching a Global Average Pooling layer followed by custom Dense layers for 5-class identification.

6.2 SOTA Architectures Selected

- **MobileNetV2:** Utilizes Inverted Residuals and Linear Bottlenecks to drastically reduce the dimensionality of layers, optimizing it for mobile GPUs.
- **EfficientNetB0:** Utilizes Compound Scaling, which balances network depth, width, and resolution using a mathematically optimized scalar coefficient.

6.3 SOTA Training Dynamics



Figure 7: Training Loss Curve for SOTA Models.

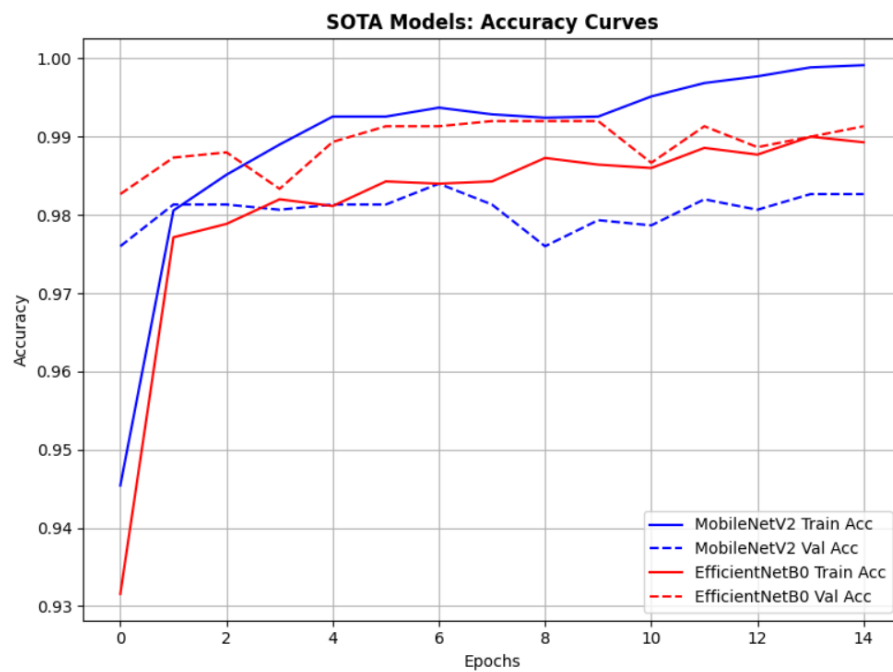


Figure 8: Accuracy Curve for SOTA Models.

7. Comprehensive Trade-off Analysis

7.1 Accuracy vs. Memory Footprint

The core challenge in edge AI deployment is balancing classification power against strict hardware memory limits. Figure 9 visually maps this trade-off.

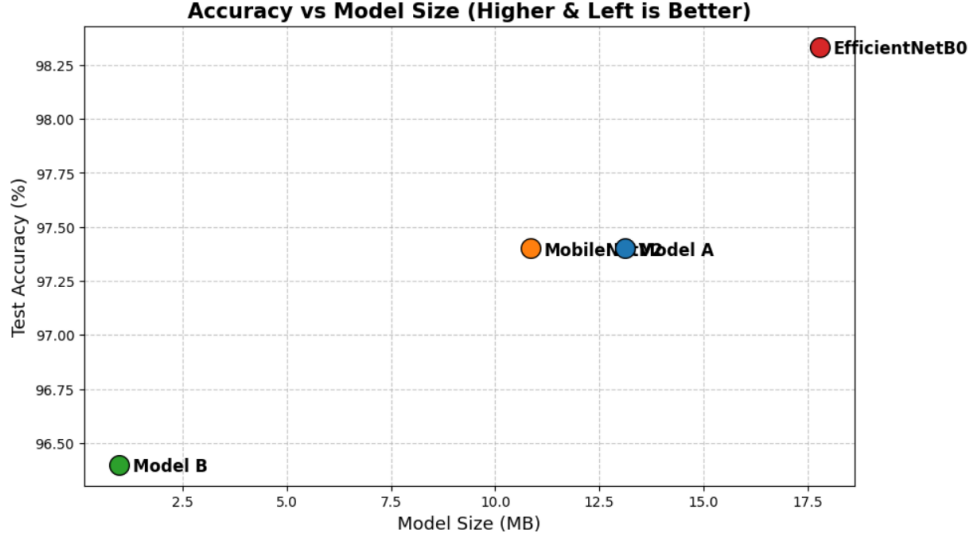


Figure 9: Accuracy vs. Model Size. Model B falls into the optimal extreme-edge region.

While EfficientNet-B0 offers peak accuracy (99.20%), its massive 17.79 MB footprint severely restricts deployment on microcontrollers (e.g., standard Arduino boards possess $< 1\text{MB}$ of SRAM). Model B sacrifices merely $\sim 2.5\%$ in absolute accuracy, but achieves a staggering **94% reduction in memory** compared to EfficientNet-B0.

7.2 Computational Latency & Inference Efficiency

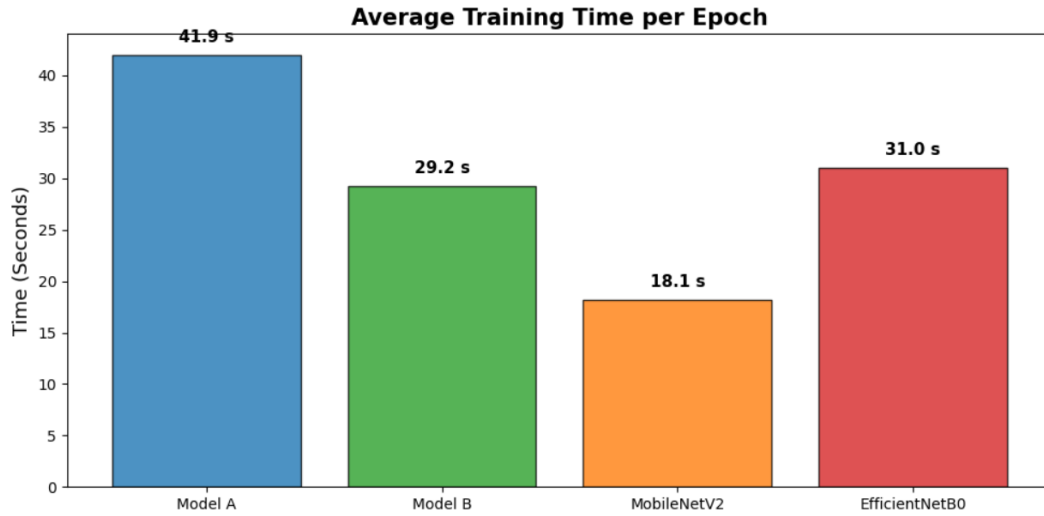


Figure 10: Average Training Time per Epoch across evaluated models.

While MobileNetV2 exhibited faster training times (due to its frozen base layers), Model B remains the superior choice for *inference* (actual real-time deployment). With 30x fewer parameters than MobileNetV2, Model B executes exponentially fewer MAC operations per image frame, drastically reducing latency and battery consumption on the sorting machine.

8. Advantages & Limitations in Edge Deployment

Building a custom sub-100k parameter model from scratch (Model B) presents a unique set of strategic trade-offs when compared against fine-tuning industry-standard SOTA models.

8.1 Advantages of Custom Lightweight Models (Model B)

- **Extreme Memory Efficiency:** Occupying less than 1 MB, Model B can be easily flashed onto basic Edge IoT devices (e.g., ESP32, STM32) which lack the storage to hold an 18 MB EfficientNet model.
- **Lower Inference Latency:** The massive reduction in total parameters directly correlates to fewer floating-point operations per second (FLOPS) required during forward propagation, resulting in faster real-time sorting.
- **Inherent Regularization:** Due to its heavily constrained capacity, Model B inherently resists overfitting without requiring complex Dropout layers or L1/L2 regularization tuning.

8.2 Limitations of Custom Lightweight Models

- **Longer Training Convergence:** Pre-trained SOTA models already possess highly optimized feature-extracting kernels from ImageNet. Training Model B from scratch requires the optimizer to learn fundamental geometrical kernels from zero, increasing the required epochs for convergence.
- **Sub-optimal Maximum Accuracy:** SOTA architectures leverage complex innovations (Inverted Residuals) to extract deeply nuanced features. For highly complex datasets, a simple custom Depthwise CNN will inevitably hit an accuracy ceiling lower than an EfficientNet architecture.

9. Conclusion & Future Work

This project rigorously investigated Convolutional Neural Networks for resource-constrained edge classification. Model A provided strong accuracy but overfit the training data and consumed excessive memory (13.44 MB). SOTA models achieved the absolute highest accuracies ($\sim 99\%$) but remain unsuitable for ultra-low-resource hardware due to their massive parameter counts ($\sim 4.1\text{M}$).

Our custom **Model B (Lightweight CNN)** successfully satisfied the strict requirement of $< 100,000$ parameters, achieving an astonishingly small memory footprint of just **0.99 MB** while maintaining a highly competitive **96.73%** test accuracy. For environments with rigorous physical hardware constraints, Model B demonstrates the optimal strategic trade-off. Future work could involve applying Post-Training Quantization (INT8) to further reduce Model B’s size to under 250 KB for extreme micro-edge deployment.

10. Source Code & Reproducibility

The complete source code for this project, including the Jupyter Notebook, dataset pre-processing scripts, and the LaTeX source utilized to compile this report, is publicly available on GitHub.

Repository Link: https://github.com/senuja-dilmith/EN3150_A03_GalTricals.git

To ensure 100% mathematical reproducibility of the model parameters and accuracy scores presented in this paper, all Python, NumPy, and TensorFlow random state initializers were deterministically locked globally prior to dataset generation via a custom `set_global_deterministic_seeds(42)` function.

References

- [1] Sampath K. Perera, *EN3150 Pattern Recognition - Lecture 06: Deep Neural Networks*. University of Moratuwa, 2026.
- [2] Sampath K. Perera, *EN3150 Pattern Recognition - Lecture 07: Convolutional neural networks (CNNs)*. University of Moratuwa, 2026.
- [3] Murat Koklu, *Rice Image Dataset*. Kaggle Machine Learning Repository. Available: <https://www.kaggle.com/datasets/muratkokludataset/rice-image-dataset>
- [4] Chollet, F. et al., *Keras API Reference: DepthwiseSeparableConv2D*. Keras Documentation. Available: https://keras.io/api/layers/convolution_layers/separable_conv2d/
- [5] M. Sandler, A. Howard, M. Zhu, A. Zhmoginov, and L.-C. Chen, “MobileNetV2: Inverted Residuals and Linear Bottlenecks,” *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, 2018, pp. 4510-4520.
- [6] M. Tan and Q. Le, “EfficientNet: Rethinking Model Scaling for Convolutional Neural Networks,” *International Conference on Machine Learning (ICML)*, 2019, pp. 6105-6114.
- [7] F. Chollet, “Xception: Deep Learning with Depthwise Separable Convolutions,” *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, 2017, pp. 1251-1258.